Rethinking Learning and Assessment

On the Three Types of Students, Three Forms of Competence, and the Use of Artificial Intelligence as a Tool for Redundancy

A Conceptual Framework for Designing More Flexible, Conscious, and Accessible Learning Systems

AI Usage: 99% (Chatgpt, Gemini, Claude)

Compiled By: Hamza Sulyman

(suahamza1@gmail.com)

Date: 10/October/2026

Abstract

Contemporary education often assumes that students should progress through a common curriculum, devote comparable forms of attention to their studies, and demonstrate competence through a relatively narrow set of assessments. Yet students differ in the time available to them, their initial familiarity with a subject, their ability to sustain structured study, and the degree to which their knowledge is already consolidated. A learning system that does not explicitly account for these differences risks introducing unnecessary friction into the educational process.

This paper proposes a conceptual framework for rethinking learning and assessment through two central concepts: redundancy and consolidation. Redundancy refers to repeated exposure, experimentation, familiarity, and the development of intuitive recognition within a domain. Consolidation refers to the organization of knowledge into more structured, explicit, and retrievable forms. The paper introduces three forms of competence—those who know, those who know that they know, and those who know that they know that they know—and uses them as an analogy for three broad student circumstances: the busy student, the student with mixed availability and motivation, and the student with substantial time and pre-existing familiarity.

The proposed framework organizes formal learning into three categories: concepts students should know, materials they should read, and tasks they should experience. It further proposes a learning cycle of reading, discussion, and practical experimentation, with artificial intelligence (AI) used particularly to increase the variety and frequency of practical engagement. Assessment is similarly divided into redundancy-oriented, mixed, and consolidation-oriented components.

Drawing on the related ideas of attention, sustainable commitment, and observable progress, the paper argues that a learning system should accommodate different starting points without lowering the standard of competence. Its objective is to make progress easier to initiate, easier to sustain, and easier to demonstrate. The framework is presented as a conceptual proposal requiring empirical testing, rather than as an established theory of learning.

Keywords: artificial intelligence, educational design, redundancy, consolidation, competence, assessment, student diversity, software engineering, attention, learning systems.

1. Introduction: The Problem Is Not Necessarily the Student

Imagine three students enrolled in the same software engineering course.

The first has family responsibilities, financial obligations, and very little free time. They need the qualification and want to complete the course, but they cannot consistently devote hours to reading, experimentation, and revision.

The second has some time available and makes a reasonable effort. They attend lectures, complete assignments, and occasionally explore subjects independently. Their engagement is neither consistently intensive nor entirely minimal.

The third has considerable time available and finds many programming concepts intuitively accessible. They recognize patterns quickly, experiment readily, and may solve problems before they can explain precisely how they arrived at a solution.

A conventional educational system may give all three students the same materials, the same deadlines, and broadly similar assessments.

A common curriculum is not inherently wrong. Students within a discipline must still meet appropriate standards. The difficulty arises when a common curriculum is mistaken for a requirement that every student must make progress through precisely the same combination of activities, attention, and effort.

The first student may need a more efficient path through essential requirements. The second may benefit from a balanced combination of structure and experimentation. The third may need greater conceptual structure to develop an intuitive understanding into explicit, reliable competence.

These differences suggest a question:

Can an educational system explicitly accommodate different student circumstances from the beginning, rather than requiring students to discover their own ways of surviving the system?

This paper proposes that it can.

The central argument is that learning should be designed around two complementary processes: redundancy and consolidation. Artificial intelligence can make repeated experimentation substantially easier, while textbooks, structured instruction, and carefully designed assessments can provide the foundations needed to organize what experimentation produces.

The objective is not to create three separate educational standards. It is to create a common learning system with multiple routes through its activities, while preserving clear expectations about what students must ultimately demonstrate.

The framework is developed through the example of a programming or software engineering student, although its underlying ideas may also be relevant to other disciplines.

2. The Conceptual Foundation: Redundancy and Consolidation

The proposed framework begins by distinguishing two forms of engagement with knowledge.

2.1 Redundancy: Becoming Familiar With a Domain

In this framework, redundancy refers to repeated encounters with concepts, problems, examples, explanations, patterns, and solutions within a particular domain.

A student learning programming might encounter the same problem through ten different implementations. They might read several explanations of an API, examine different approaches to a database query, or repeatedly modify a working application.

Through these encounters, the student develops familiarity with the domain.

They may begin recognizing which approaches are likely to work, which errors are common, and which patterns appear repeatedly. Some of this knowledge may initially be intuitive. The student may recognize a good solution without being able to explain every reason it is good.

Redundancy therefore includes forms of knowing that are not yet fully articulated or systematically organized.

It is not synonymous with meaningless repetition. Its educational value depends on the variety, relevance, and quality of the encounters involved.

For example, generating ten identical solutions with slightly different variable names may offer little additional value. Comparing ten meaningfully different solutions to the same problem may reveal alternative algorithms, trade-offs, abstractions, and implementation strategies.

The purpose of redundancy is to increase familiarity, recognition, and opportunities for discovery.

2.2 Consolidation: Organizing Knowledge Into Structure

Consolidation refers to the development of structured, explicit, and retrievable knowledge.

A student moves beyond recognizing a solution and begins understanding the principles that explain why it works. They can identify relevant concepts, explain relationships, recognize constraints, and apply knowledge in a deliberate manner.

In programming, this might mean understanding not merely how to write a database query, but why a particular query is appropriate, what its limitations are, and how its performance changes under different conditions.

Consolidation provides a structure through which familiarity becomes more organized and dependable.

Reading technical textbooks, receiving structured explanations, studying underlying principles, and reconstructing solutions can all contribute to this process.

Redundancy and consolidation are complementary. Familiarity without sufficient structure can leave a student with fragmented intuitions. Structure without sufficient exposure and experimentation can leave knowledge brittle, difficult to apply, or disconnected from practical situations.

A useful shorthand is:

• Redundancy: Encounter the domain repeatedly and in different forms.

• Consolidation: Organize what has been encountered into a coherent structure.

Neither process should be expected to replace the other.

2.3 A Note on Terminology

The word redundancy is used here in a particular conceptual sense. In information systems, redundancy can mean duplicated information. In education, related processes are often discussed using terms such as repeated exposure, retrieval practice, varied practice, and exposure to multiple examples.

The proposed framework uses redundancy as an umbrella term for repeated and varied engagement with a domain, particularly when that engagement encourages familiarity and intuitive recognition.

Likewise, consolidation is used broadly to describe the structuring of knowledge. It should not be assumed to be identical to the narrower psychological meaning of memory consolidation.

These distinctions matter because the framework is intended to provide a practical design language, not to claim that its terminology replaces established educational or cognitive science concepts.

3. Three Forms of Competence

The relationship between redundancy and consolidation provides the basis for a second distinction: three forms of competence within a domain.

These categories describe possible configurations of knowledge rather than fixed classes of people.

3.1 Type A: Those Who Know

Type A competence is characterized by substantial familiarity and relatively limited consolidation.

A person may recognize patterns, understand examples intuitively, and have a general sense of how a domain works. However, their knowledge may be difficult to explain systematically or retrieve as a set of explicit principles.

A programmer might recognize that a particular implementation is inefficient without being able to explain the relevant complexity analysis. They might know how to solve a recurring problem because they have encountered similar problems before, even if they cannot fully articulate the underlying reasoning.

This is not an absence of knowledge. It is a form of knowledge that may not yet be sufficiently structured.

In the language of the framework, Type A competence is relatively redundancy-heavy.

3.2 Type B: Those Who Know That They Know

Type B competence represents a more balanced relationship between familiarity and consolidation.

The individual recognizes patterns and can also explain a meaningful portion of the principles behind them. They have accumulated experience while developing a structure through which that experience can be interpreted.

A programmer at this level might recognize several ways to solve a problem, explain the differences between them, and choose an appropriate implementation under familiar constraints.

The balance is not necessarily an exact fifty-fifty division. It is a conceptual midpoint between intuitive familiarity and highly structured understanding.

3.3 Type C: Those Who Know That They Know That They Know

Type C competence represents highly consolidated knowledge.

The individual possesses a strong conceptual structure within the domain. They can explain principles, connect ideas, reason through unfamiliar situations, and make deliberate decisions based on an organized understanding.

Their competence is sufficiently developed that they may approach familiar problems with confidence and little conscious hesitation.

The playful formulation—those who know that they know that they know—captures the confidence that can accompany deep familiarity and structure.

However, confidence should not be confused with infallibility. Even highly competent individuals can possess blind spots, encounter unfamiliar problems, or be mistaken about the limits of their knowledge.

The defining feature of Type C is therefore not the absence of doubt, but the degree to which knowledge has become organized, explicit, and usable.

3.4 The Three Types as a Continuum

The three types are best understood as conceptual positions rather than permanent identities.

A person might demonstrate Type C competence in one programming language, Type B competence in database design, and Type A competence in cloud infrastructure.

A beginner might also possess extensive familiarity with a subject before receiving formal instruction, while an experienced practitioner might discover that an unfamiliar subdomain requires considerable consolidation.

Competence is domain-specific and can vary across tasks.

The framework therefore does not require educators to classify students permanently. Its purpose is to help identify which kinds of learning activity may be useful at a particular stage.

4. Three Student Circumstances

The competence analogy becomes more useful when applied to the practical circumstances of students.

The proposed categories are deliberately simple. They are not psychological diagnoses, moral judgments, or explanations of why a person behaves in a particular way.

Their purpose is to acknowledge that students arrive with different amounts of time, attention, and prior familiarity.

4.1 The Busy Student

The busy student has limited time available for study.

They may be working, caring for family members, managing financial responsibilities, pursuing other ambitions, or simply allocating their time elsewhere. An educational framework does not need to determine whether these circumstances are justified.

The relevant fact is that their available time is limited.

Such a student may prioritize obtaining a qualification, satisfying formal requirements, and completing the course within the constraints of their circumstances.

They may also begin with relatively little familiarity with the subject. This combination creates a particular challenge: the student needs to make progress while having limited opportunities for repeated experimentation and extended study.

A useful educational system should therefore make the essential requirements explicit, provide a clear minimum path through them, and allow the student to use AI to increase the productivity and variety of limited practical sessions.

The objective is not to promise mastery without effort. It is to avoid requiring unnecessary effort merely to determine what effort is required.

4.2 The Student With Mixed Availability and Engagement

The second student has some time and makes a reasonable effort, but their availability and engagement fluctuate.

They may complete assignments, attend lectures, and follow the curriculum without consistently pursuing additional material. Sometimes they have time for deeper exploration; at other times, they simply complete the next requirement.

This student benefits from a system that supports both structured progression and optional experimentation.

When time is available, they can explore alternative solutions, read additional explanations, or pursue a more demanding project. When time is limited, they can return to the formal requirements without needing to reconstruct the entire learning plan.

The educational system should accommodate changes in availability without treating every interruption as a failure of commitment.

4.3 The Student With Substantial Time and Built-In Redundancy

The third student has considerable time available and may possess an unusually high degree of initial familiarity with the domain.

Concepts feel intuitive. Examples are quickly recognizable. Practical tasks may appear straightforward, even when the student has not yet studied the formal principles behind them.

This student may benefit from redundancy almost effortlessly because they enjoy exploring the subject or readily recognize patterns.

However, familiarity alone is not sufficient for every educational objective.

An intuitive programmer may produce working applications without understanding security implications, architectural constraints, computational complexity, or the principles needed to maintain a large system.

They still need structure.

For this student, reading, explanation, formal reasoning, and deliberate reconstruction may provide greater marginal value than additional repetitions of tasks they already perform comfortably

4.4 Why These Are Circumstances, Not Boxes

The framework does not assume that every student belongs permanently to one category.

Time availability can change. Familiarity can differ across subjects. Motivation can fluctuate. A student who has little time for one semester may have considerably more during another.

The categories are useful because they make different circumstances visible without requiring educators to judge students' personal lives.

The design question is not which type of student deserves a different standard. It is which combination of learning activities makes meaningful progress most accessible to that student.

5. The Central Design Principle: Make Progress an Explicit Feature

The three student circumstances lead to a practical design principle.

A student should not have to discover accidentally that there is a workable way to progress through a course under their particular constraints.

If the curriculum requires reading, practical tasks, and familiarity with essential concepts, these requirements should be stated explicitly.

If AI is permitted and encouraged for experimentation, that permission should also be explicit.

If certain requirements are essential for certification while other activities are optional opportunities for deeper learning, the distinction should be visible from the beginning.

This creates a more conscious learning system: students understand what is required, what is encouraged, and how different activities contribute to their progress.

The principle connects with two related ideas developed in the companion papers The Baseline and the Long Game and Proof of Movement.

5.1 Design for the Base Case

An ambitious study plan often assumes ideal conditions: consistent motivation, uninterrupted time, and the ability to sustain demanding work every day.

A more durable system begins with a slower, realistic path.

For a busy student, this might mean defining the essential concepts, minimum assigned readings, and core practical tasks that can be completed under ordinary constraints.

Additional activities remain valuable, but the student does not have to depend on an ideal schedule for the entire plan to remain viable.

The principle is not to make learning effortless. It is to ensure that the plan can survive ordinary inconsistency.

5.2 Require Proof of Movement

A learning system should make progress observable.

Instead of relying exclusively on intentions such as wanting to become a better programmer, students can record concrete evidence of engagement:

• Concepts they have encountered and can discuss.

• Chapters or sections they have read.

• Practical tasks they have attempted.

• Working solutions they have generated or modified.

• Problems they have investigated and explained.

This evidence does not automatically prove mastery. It establishes a record of activity that can be combined with assessments of competence.

The distinction is important: a student can demonstrate movement without yet demonstrating mastery, and a well-designed system should recognize both facts.

The combination of a realistic baseline and recurring evidence of movement offers a practical foundation for a flexible learning process.

6. The Proposed Learning Architecture

The framework proposes the following learning cycle:

Read → Discuss (Lecturing/Teaching) and Code → Code → Discuss (Lecturing/Teaching) and Code → Repeat

This sequence is not intended as a rigid timetable. It represents the recurring interaction between structured knowledge, guided explanation, and practical experience.

Three activities are central:

• Reading provides a principal source of structure.

• Discussion and teaching help students interpret, connect, and clarify knowledge.

• Coding provides repeated opportunities to experiment with that knowledge.

The activities can overlap and recur in different orders, but each has a distinct contribution to the proposed system.

6.1 Reading: The Structural Foundation

The framework gives formal reading a prominent role.

A curated textbook or technical text offers a sustained account of a subject. It provides terminology, explanations, examples, relationships, and a structure that individual practical tasks may not provide on their own.

A student who encounters programming concepts exclusively through isolated AI-generated examples may acquire useful fragments of knowledge without understanding how those fragments fit together.

A textbook can provide a more continuous account of the discipline.

The proposal therefore distinguishes reading from merely consuming material about a subject. Listening to an audiobook, watching a video, or asking an AI assistant to summarize a chapter may be useful, but these activities are not identical to reading the assigned text.

They may supplement reading, but should not automatically count as its equivalent when the formal requirement is specifically to read.

Reading as an explicit requirement

An educator should specify:

• Which textbooks or technical texts are assigned.

• Which chapters or sections are required.

• Whether a text should be read in full or selectively.

• What evidence of completion is expected.

The framework makes an additional distinction: completing an assigned reading task does not necessarily mean that the student has understood everything in the text.

A student may read a difficult chapter, encounter unfamiliar material, and continue without fully resolving every question.

That reading can still be recorded as completed, even though understanding remains incomplete.

This is a proposal about tracking educational activity, not a claim that reading without comprehension is sufficient for competence. The distinction allows reading requirements and mastery requirements to be assessed separately.

The student can honestly say, "I have read this chapter," without being required to pretend, "I have mastered everything in this chapter."

6.2 Discussion and Teaching: Guided Interpretation

The educator occupies an important position between reading and experimentation.

A lecturer can clarify difficult concepts, identify misconceptions, connect ideas across chapters, and help students determine when they need further explanation or additional practice.

Some students may already recognize a concept intuitively but struggle to explain it. Others may understand a definition but lack sufficient practical familiarity to recognize when it applies.

The appropriate intervention differs.

For one student, another explanation may provide the missing structure. For another, repeated practical examples may be more useful. A third may benefit from both.

An effective educator can help identify these differences and guide the student's next activity accordingly.

This role is not eliminated by AI. AI can generate explanations and examples at scale, but the educator remains important in designing the curriculum, judging the significance of misconceptions, and ensuring that educational activities remain aligned with the intended learning outcomes.

6.3 Coding and Experimentation: The Redundancy Engine

Practical experimentation provides opportunities to encounter the domain repeatedly.

A programming student might implement the same feature using different techniques, inspect alternative architectures, modify working code, or compare several solutions to a single problem.

AI makes these activities considerably easier to generate and explore.

A student can request several implementations of a task, ask for variations in style or complexity, investigate different approaches, and test the results. They can use AI to produce examples that would otherwise require considerable time to prepare.

The important distinction is that AI is encouraged as a tool for experimentation, not merely permitted as a last resort when a student cannot solve a problem independently.

This proposal rejects the assumption that every use of AI must follow a single prescribed tutoring method. Students should have room to explore, generate, modify, and compare solutions in ways that suit their circumstances.

However, the resulting code should be tested and examined. AI-generated solutions may be incorrect, insecure, inefficient, or unsuitable for the task.

The objective is not to maximize the volume of generated code. It is to increase meaningful exposure to the domain.

7. A Three-Part Formal Curriculum

The proposed learning architecture becomes operational through a curriculum divided into three explicit categories:

- Know these.
- Read these.
- Experience these tasks.

Each category has a different purpose and should be identified separately in the course materials.

7.1 Category One: Know These

This category contains a carefully curated list of important terms, concepts, technologies, and recurring ideas within the domain.

For an introductory WordPress course, the list might include:

HTML/CSS, JavaScript, PHP, MySQL, HTTP, WordPress Core, Template Hierarchy, The Loop, themes, plugins, databases, and local development.

The list should be sufficiently selective to remain manageable. It is not intended to reproduce the entire syllabus.

Students should be able to talk or write about the listed concepts. Depending on the student's current stage, their knowledge may range from intuitive recognition to explicit, structured explanation.

The formal requirement is that these concepts should not remain entirely unfamiliar.

The educator can establish a minimum standard for each concept, while allowing the depth of understanding to develop through reading, discussion, and experimentation.

7.2 Category Two: Read These

This category specifies the texts that students are expected to read.

For the WordPress example, the initial curriculum might include:

• Professional WordPress: Design and Development

• HTML and CSS: Design and Build Websites

• PHP & MySQL: Server-side Web Development

• JavaScript and jQuery

These are illustrative selections from the proposed curriculum. An actual course should verify the editions, technical currency, suitability, and relevance of its assigned materials.

The purpose of this category is to establish a formal reading path through the subject.

Reading completion and comprehension should be tracked separately. A student may complete the assigned pages without mastering them, but the course should still provide appropriate opportunities to test whether essential concepts have been understood.

This distinction prevents two requirements from being confused: having engaged with the prescribed material and being able to demonstrate the competence that the material is intended to support.

7.3 Category Three: Experience These Tasks

This category identifies practical activities through which students encounter and experiment with the domain.

For an introductory WordPress course, the sequence might include:

• Build basic webpages.

• Build a small JavaScript/API project.

• Build a PHP application.

• Install WordPress locally.

• Explore WordPress Core.

• Build a basic WordPress website.

The tasks should be specific enough to establish what students are expected to attempt, but flexible enough to support multiple implementations.

AI use should be explicitly encouraged where appropriate. Students may generate working examples, request alternative implementations, modify solutions, compare approaches, and explore variations.

The primary requirement at the experimentation stage is that students actually engage with the tasks. Whether that engagement has produced reliable competence is a separate question that the assessment system must address.

7.4 Formal and Informal Learning

These three categories constitute the formal curriculum.

The formal curriculum specifies what students are required to know, read, and experience.

The informal curriculum includes other activities that students may undertake:

• Listening to audiobooks.

• Watching programming tutorials.

• Reading additional textbooks.

• Exploring documentation and technical websites.

• Inspecting other people's code.

• Experimenting with additional projects.

• Discussing ideas with peers.

• Using AI to explore questions beyond the syllabus.

These activities can enrich learning, but they should not obscure the formal requirements.

The distinction allows students to explore freely without having to guess which activities are mandatory.

It also gives educators a clear basis for distinguishing required work from additional opportunities for development.

8. The Proposed Distribution of Learning Activities

The framework suggests an approximate distribution of learning activity across three functions.

Learning activity	**Proposed allocation**	**Primary function**
Reading	33%	Consolidation
Discussion and teaching	33%	Consolidation and redundancy
Coding and experimentation	33%	Redundancy

The percentages are conceptual starting points, not experimentally established optimal ratios. Because three equal shares total approximately 99% when rounded to whole percentages, an implementation could use 33%, 34%, and 33%.

The distribution is intended to emphasize the presence of all three activities rather than prescribe an inflexible schedule.

8.1 Reading and Consolidation

Reading provides a sustained structure through which concepts can be encountered in relation to one another.

It establishes the formal foundation around which practical experience can be organized.

This does not mean that reading always produces consolidation or that consolidation occurs only through reading. Rather, the framework assigns reading the primary responsibility for providing a continuous conceptual structure.

8.2 Discussion and the Educator's Judgment

Discussion can support either redundancy or consolidation, depending on what the student needs.

An educator might explain a concept through several different examples to build familiarity. Alternatively, the educator might formalize an intuitive idea, identify its underlying assumptions, or connect it to a broader theory.

Discussion is therefore the flexible middle of the learning system.

The proposed one-third allocation should not prevent educators from increasing or decreasing discussion time in response to the course's requirements.

8.3 Coding and Redundancy

Coding provides opportunities to encounter concepts through action.

Students can experiment with implementations, investigate errors, and discover differences between solutions.

AI is particularly valuable here because it can generate numerous variations quickly, enabling students to encounter more examples than would ordinarily be practical within the same period.

However, this advantage depends on the quality of the activities. Repeated exposure to incorrect examples or unexamined code may reinforce misconceptions rather than improve competence.

The educator should therefore curate the tasks, while students should test and examine the resulting work.

The broader principle is simple: a student should not be expected to consolidate knowledge continuously. There must also be room to explore, encounter, and experiment.

9. Artificial Intelligence as a Tool for Redundancy

AI changes the practical economics of experimentation.

Before widely accessible generative AI, producing multiple working implementations of a programming task might require substantial time from the student or educator.

An AI assistant can generate alternative implementations, explain their differences, propose modifications, and help construct new variations.

This makes it possible to design redundancy directly into the curriculum.

9.1 Generating Multiple Solutions

Suppose students are learning how to validate user input in a PHP application.

They might generate several implementations:

• A straightforward conditional implementation.

• A reusable validation function.

• An object-oriented approach.

• An implementation using an existing validation library.

• A version with additional error handling.

• A version designed to handle unexpected input more safely.

The student can compare these approaches and explore how each responds to different requirements.

The activity creates repeated exposure to the problem while revealing meaningful differences between solutions.

9.2 Combining and Modifying Solutions

Students can then modify the examples, combine useful ideas, and create alternative implementations.

This may expose them to abstractions or design patterns they would not have discovered through a single attempt.

For example, they might take the readability of one solution, the error-handling approach of another, and the modularity of a third to produce a new implementation.

The educational value lies in the encounter with alternatives and the subsequent process of selection, testing, and reconstruction.

Simply combining pieces of code does not guarantee understanding. Students must still verify that the resulting implementation works and satisfies the relevant requirements.

9.3 AI Use Should Be Explicit

The framework proposes that course materials state clearly where AI is encouraged, what kinds of experimentation it supports, and which outputs must be verified.

Students should not have to guess whether using AI for practical exploration is an accidental loophole or a deliberate component of the course.

AI can be treated as a normal instrument of learning, much as documentation, development tools, and code editors are treated as ordinary instruments of programming.

This does not eliminate the need for academic integrity. The distinction is between using AI to complete authorized experimentation and misrepresenting AI-generated work as evidence of a competence that has not been demonstrated.

The system should specify the distinction in advance.

9.4 The Limits of AI-Generated Redundancy

More examples do not necessarily produce better learning.

AI may generate plausible but incorrect explanations, duplicate implementations, insecure code, or solutions that work only under narrow conditions.

A student who generates ten solutions without inspecting them may accumulate outputs without acquiring meaningful familiarity.

The framework therefore requires that redundancy be purposeful, varied, and connected to the intended domain.

AI should reduce the cost of experimentation, not eliminate the responsibility to examine the results.

10. Rethinking Assessment: What Should Be Measured?

If learning includes both redundancy and consolidation, assessment should provide opportunities to demonstrate both.

A system that tests only formal explanations may fail to recognize practical familiarity. A system that tests only whether code executes may fail to establish whether the student understands the underlying principles.

The proposed assessment model therefore contains three components:

Assessment component	**Proposed weight**	**Primary purpose**
Redundancy-oriented assessment	33%	Familiarity and exploration
Mixed assessment	34%	Recognition and structured interpretation
Consolidation-oriented assessment	33%	Deliberate reasoning and integration

The proposed allocation is a starting hypothesis. It should be adjusted according to the learning outcomes, the stakes of the assessment, and evidence from actual implementation.

10.1 Component One: The Redundancy Check

The first component evaluates a student's engagement with multiple examples and approaches.

For a programming task, students might be asked to generate ten working solutions using AI.

They would be expected to produce or document the solutions and demonstrate that the implementations have been tested against the specified requirements.

The assessment might consider:

• Whether the required number of meaningful variations was produced.

• Whether the solutions address the same underlying problem.

• Whether the solutions work under the stated test conditions.

• Whether the student can identify differences between the implementations.

The primary purpose is to assess exposure, experimentation, and familiarity rather than demand a comprehensive theoretical explanation of every solution.

However, merely submitting ten AI-generated code blocks should not automatically earn full marks. The solutions must satisfy the task's requirements, and the assessment must be transparent about what counts as a meaningful variation.

This component can recognize practical engagement without pretending that engagement alone proves mastery.

10.2 Component Two: The Redundancy–Consolidation Check

The second component asks the student to select a solution that stands out and explain why.

For example:

Which of the ten solutions impressed you most? Reproduce it and explain what stood out to you. Was it simpler, more sophisticated, more readable, more efficient, or more robust than the alternatives?

This question creates a bridge between familiarity and structured judgment.

The student must move beyond generating examples and identify a meaningful property of one implementation.

The explanation need not be unnecessarily elaborate, but it should be grounded in observable features of the code.

A student might identify that one solution separates responsibilities more clearly, handles invalid input more reliably, or avoids unnecessary complexity.

The educator can evaluate whether the selected solution has been represented accurately and whether the explanation demonstrates a reasonable understanding of its distinguishing features.

The component therefore assesses both exposure to alternatives and the ability to organize some of that exposure into explicit knowledge.

10.3 Component Three: The Consolidation Check

The third component asks students to integrate ideas from multiple solutions.

For example:

A. Combine two of the existing solutions to create a new working implementation.

B. Attempt to combine useful ideas from three or more solutions into another implementation.

Students should explain which elements they selected, why those elements belong together, and how they verified the resulting code.

This activity requires more than producing variations. It requires the student to make decisions about compatibility, structure, and functionality.

A successful implementation may demonstrate an ability to recognize useful components, reason about their relationships, and organize them into a coherent result.

However, the task should not reward combination for its own sake. A simpler solution may be better than an unnecessarily complicated hybrid.

Students should be permitted to conclude that two solutions should not be combined if their approaches are incompatible or if the combination would reduce clarity or reliability.

The assessment should reward sound judgment, not the mere quantity of merged code.

11. Matching Assessment to the Learning Process

A central principle of the proposed framework is that assessment should correspond to the learning activities that students have been given opportunities to perform.

If the curriculum explicitly encourages students to explore multiple AI-generated implementations, it is reasonable to assess their ability to work with multiple implementations.

If the curriculum assigns formal reading, students should also have opportunities to demonstrate the understanding that the reading is intended to develop.

If the course expects students to construct reliable applications, the assessment must ultimately establish that their work meets the required functional and technical standards.

The three assessment components should therefore be integrated rather than treated as unrelated examinations.

The redundancy-oriented component asks whether students have encountered and experimented with alternatives.

The mixed component asks whether they can recognize and explain meaningful differences.

The consolidation-oriented component asks whether they can reason about, select, integrate, and validate those alternatives.

Together, these components provide a broader picture of student competence.

11.1 A Proposed Assessment Rubric

An implementation could use the following illustrative rubric.

Component	**What is evaluated**	**Evidence of success**
Redundancy	Breadth and quality of experimentation	Multiple meaningful, tested solutions
Mixed	Recognition and explanation	Accurate selection and reasoned comparison
Consolidation	Integration and deliberate judgment	A coherent, tested implementation with justified design choices

The rubric should be adapted to the course and the complexity of the task.

For an introductory course, the emphasis might be on basic functionality, meaningful variation, and clear explanations. For an advanced course, the same structure might evaluate security, architecture, scalability, performance, and maintainability.

The weights should not be mistaken for a universal grading formula. Their purpose is to establish a starting point for designing assessments that recognize different forms of competence.

12. Making Room for the Three Students

The framework becomes most useful when its components are combined into a system that supports different circumstances without creating separate educational standards.

12.1 The Busy Student: A Clear Minimum Path

The busy student benefits from explicit requirements.

They can see the essential concepts, assigned readings, practical tasks, and assessment criteria from the beginning.

They can use AI to generate alternative solutions, investigate difficult implementation details, and make practical experimentation more efficient.

Their progress can be recorded as they complete the formal requirements.

The system does not promise that the student can become highly competent without sufficient practice. Rather, it reduces unnecessary uncertainty about what needs to be done and allows limited study time to be directed toward defined objectives.

Where an assessment requires independent demonstration of competence, that requirement remains in place.

12.2 The Student With Mixed Availability: Flexible Progression

The second student benefits from a system that accommodates changing circumstances.

When they have time, they can read beyond the minimum, generate more examples, and explore additional implementations.

When they have less time, they can return to the core curriculum and continue from their recorded point of progress.

The framework avoids treating every variation in engagement as a need to restart the learning process.

A visible record of completed readings, attempted tasks, and demonstrated competencies can help the student resume work without unnecessary friction.

12.3 The Student With Substantial Time and Familiarity: Greater Depth

The third student can use the same system to pursue greater depth.

They may generate numerous solutions, compare architectural alternatives, study advanced texts, or design more demanding projects.

The curriculum can also require them to explain their intuitive decisions, identify assumptions, and justify their choices.

This helps turn familiarity into more structured competence.

The framework does not require this student to spend unnecessary time repeating tasks they have already mastered. Instead, it provides opportunities to demonstrate existing competence and direct additional effort toward more demanding objectives.

12.4 A Common Standard, Multiple Routes

These accommodations should not imply that students receive different qualifications for demonstrating different levels of competence.

A course may allow different routes through practice and preparation while maintaining common requirements for its essential learning outcomes.

For example, one student might generate several implementations with AI before studying the underlying theory, while another begins with the textbook and develops implementations afterward.

Both may ultimately be required to demonstrate that they can construct, test, explain, and maintain an appropriate solution.

The flexibility concerns the learning process, not the legitimacy of the required outcome.

13. Implications for Educators and Institutions

The framework requires more than giving students access to an AI chatbot.

It requires educators to make deliberate decisions about curriculum design, assessment, and the evidence used to evaluate competence.

13.1 Curriculum Design Becomes More Explicit

Every course should identify its essential concepts, assigned reading, practical tasks, and expected learning outcomes.

This makes the structure of the course more visible and reduces the burden on students to infer what matters most.

13.2 Assessment Must Distinguish Activity From Achievement

A completed reading is evidence of reading activity, not necessarily comprehension.

A generated program is evidence of producing an artifact, not necessarily independent programming competence.

A student explanation is evidence of understanding only to the extent that it is accurate and appropriate to the task.

The framework becomes more credible when these distinctions are explicit.

13.3 AI Use Requires Verification and Transparency

Where AI is permitted, students should be taught how to test outputs, identify errors, and document their use when necessary.

Instructors should establish clear expectations about which tasks may use AI, which require independent performance, and how students should demonstrate their own competence.

These requirements are particularly important when assessments have implications for certification, professional practice, or public safety.

13.4 Educators Need Not Adopt the Entire Model at Once

An institution could begin by redesigning a single course module.

It might identify a small set of essential concepts, assign selected reading, introduce AI-supported experimentation, and pilot the three-part assessment model.

Student outcomes could then be compared with those from the existing approach.

Such a pilot would help determine whether the framework reduces unnecessary friction while preserving or improving competence.

14. Limitations and Research Questions

The framework is a conceptual proposal. Its usefulness should be investigated rather than assumed.

Several questions remain open.

First, how should redundancy be measured? The number of examples encountered may be easy to record, but the educational value of those examples depends on their variety, accuracy, and relevance.

Second, how should consolidation be measured? An explanation may appear sophisticated without demonstrating reliable understanding. Assessments need clear criteria for evaluating reasoning, transfer, and the ability to work independently when required.

Third, do the proposed assessment weights work? The suggested distribution of approximately one-third for each component is a design hypothesis. Different disciplines and learning outcomes may require different allocations.

Fourth, does AI-supported experimentation improve learning outcomes? AI makes it easier to generate multiple solutions, but it may also encourage superficial acceptance of outputs. Research should investigate when AI-supported redundancy contributes to durable competence and when it merely increases the volume of work produced.

Fifth, can the framework generalize beyond programming? Its three-part curriculum may be adaptable to other disciplines, but the specific meaning of redundancy, consolidation, and practical experimentation will vary. Evidence is needed before assuming that the same implementation will work equally well in every field.

Sixth, does the system genuinely reduce friction? A transparent curriculum may make progress easier to initiate, but it does not eliminate the time, attention, or effort required for learning. The relevant question is whether it helps students use those resources more effectively.

14.1 A Possible Pilot Study

A preliminary study could implement the framework in an introductory programming course.

Students could receive a curriculum organized around essential concepts, assigned reading, and AI-supported practical tasks. Assessments could include redundancy-oriented, mixed, and consolidation-oriented components.

Researchers could examine:

- Completion of formal curriculum requirements.
- Performance on practical and conceptual assessments.
- Students' ability to explain and modify AI-generated solutions.
- Retention of knowledge after a delay.
- Performance on unfamiliar problems.
- Student perceptions of clarity and friction.
- Differences in outcomes among students with different levels of available study time and prior familiarity.

A comparison with a conventional course design could help establish whether the proposed structure offers measurable advantages.

Importantly, the evaluation should not assume that generating more solutions necessarily means learning more. It should examine the quality of the resulting competence.

15. Conclusion: Designing a System That Makes Progress Easier

The central proposal of this paper is that learning should be designed with greater explicitness about the different activities through which competence develops.

Students need opportunities to become familiar with a domain. They also need opportunities to organize that familiarity into structured knowledge.

Redundancy and consolidation provide a conceptual distinction through which these activities can be designed and assessed.

The three forms of competence illustrate how familiarity and structure may be distributed differently across individuals and domains. The three student circumstances demonstrate why a learning system should accommodate differences in available time, engagement, and initial familiarity without treating those differences as moral defects.

The proposed curriculum makes its requirements visible through three categories: know these, read these, and experience these tasks.

Within this structure, reading provides an important foundation, educators guide interpretation, and practical experimentation creates opportunities for repeated engagement. AI can substantially reduce the cost of generating and exploring alternative solutions, making redundancy a deliberate component of the learning process rather than an accidental by-product.

Assessment can then evaluate three complementary dimensions: familiarity with alternatives, the ability to interpret and compare them, and the ability to integrate ideas into a coherent solution.

The resulting system does not promise mastery without effort. It seeks to make the relationship between effort and progress clearer.

A student should know what is required, understand how different activities contribute to learning, and have an explicit path forward even when their circumstances are less than ideal.

This brings the framework back to a broader principle: educational systems should not depend entirely on students discovering the right way to study through trial and error. They should make the structure of progress visible, provide multiple opportunities to engage with the subject, and create reliable ways to demonstrate what has been learned.

The goal is not to remove all friction from education. Some friction is inherent in difficult and meaningful work. The goal is to remove avoidable friction: unnecessary ambiguity, poorly specified requirements, inflexible study patterns, and assessments that recognize only a narrow portion of competence.

If AI can make experimentation more accessible, structured reading can provide a foundation, and assessment can distinguish activity from achievement, then a more flexible learning system becomes a plausible design objective.

Whether the proposed arrangement produces better outcomes remains an empirical question. But it is a question that can be investigated through deliberate implementation rather than postponed until every aspect of the framework has been accepted by established institutions.

The starting point is straightforward:

Make the requirements explicit. Make experimentation accessible. Preserve the need for structured understanding. Assess both familiarity and competence. Make progress visible.

A learning system designed around these principles may offer a more practical way for different students to pursue meaningful educational outcomes under different circumstances.

References and Conceptual Foundations

The following working papers provide the conceptual foundations from which this proposal is developed:

• The Two Attention Threads: Redundancy and Consolidation in Learning. Unpublished conceptual working paper, 2026.

• The Baseline and the Long Game: Solving for the Base Case Before You Commit. Unpublished conceptual working paper, 2026.

• Proof of Movement: On Commitment, Time, and the Goals We Keep Postponing. Unpublished conceptual working paper, 2026.

These papers inform the framework's treatment of attention, structured and repeated engagement, realistic commitment, and observable progress. They are presented as the author's conceptual foundations rather than as independently validated empirical evidence.

The present paper likewise proposes a framework for further development and testing. It does not claim that the proposed categories, allocations, or assessment procedures have already been validated through controlled educational research.

APPENDIX: A POEM

The Time Spent Learning

You begin with the question of learning.
Not the learning itself,
but the question of it.

How long should one sit?
What makes a lesson stay?
Is repetition enough?
Should the hand write
while the eyes read?
Should the eyes read
while the voice speaks?

You look into these things.

There are methods.
There are better methods.
There are people who have compared them.
There are people who have compared
the people who compared them.

You read about memory.

You learn that memory
is not quite what you thought it was.

You learn about forgetting,
and spacing,
and attention,
and recall.

You make a note of this.

You make another note
to return to the first note.

By then the morning has gone.

Still, something has been learned.

That is the difficulty.

Something has been learned.

Only not the thing
you sat down to learn.

You were going to learn the words.

Instead, you learned
how words are learned.

You were going to read the chapter.

Instead, you read
about reading chapters.

You were going to practise.

Instead, you became
quite interested
in what makes practice effective.

It is not nothing.

Of course it is not nothing.

One must know how to begin.

One must know what helps.

One must know when to stop
thinking about beginning.

And perhaps this is the part
that requires the most learning:

to notice that one is still

learning about learning.

So you close the article.

You put away the notes.

You decide that tomorrow
you will simply learn.

Which is a good decision.

You spend a little time
thinking about how good
the decision is.

Then you wonder
whether thinking about learning
has itself become a kind of learning.

It probably has.

You make a note of that.

And by then the morning
has gone.

You make a note of that.

And by then the morning
has gone.

Still, there is the afternoon.

There is time enough.

You sit down again
with the intention
of doing the thing itself.

No more preparation.

No more methods.

No more reading
about memory.

The book is open.

The first line is there.

You read it once.

Then again.

You wonder whether reading it twice
is repetition
or merely reading it twice.

You read it a third time
to find out.

It seems clearer.

Perhaps this is working.

You continue.

After a while
you begin to wonder
whether you are remembering
because you understand
or understanding
because you remember.

You stop.

Not for long.

Just long enough
to think about it.

There are probably studies
about this.

You could look them up.

You do not.

That is progress.

You continue reading.

A word you do not know
appears in the next line.

You could simply learn the word.

But first you wonder
whether looking it up immediately
is better than trying to recall it later.

There is probably a method
for this too.

You already know that.

You also know
what happened last time.

So you do not look it up.

You try to remember.

You cannot.

You look it up.

Now you know the word.

Or you know

that you have looked it up.

There is a difference.

You have read about that difference.

Somewhere.

You remember the article.

Not the word.

You go back.

The word is still there.

So you learn it.

At last.

This, surely,
is the thing.

Not the article,
not the method,
not the question of whether
the method is better than another method,

but this small thing
now sitting in the mind
where it was not sitting before.

You read the next word.

Then the next.

And soon enough
you begin to think
that perhaps this is what learning is:

a long interruption
of thinking about learning
by the occasional act
of learning.

Which is an interesting thought.

You write it down.

You look at what you have written.

You wonder whether writing down
an observation about learning
is itself part of learning.

It might be.

You wonder whether
you should continue.

You probably should.

You look back at the page.

The first word is still there.

You have been sitting here
for some time.

The book is open.

The notes are beside it.

The method has been considered.

The memory has been examined.

The process has been observed.

The word has been learned.

And the time spent learning about learning

has not been the time spent learning.

Though some of it was.

And some of that
was spent deciding
which part was which.

Which is another thing
worth understanding.

Perhaps.

You begin to think about it.

.....

.....

...

...
..

The time spent learning about learning
is not the time spent learning.
It is also not quite
the time spent not learning.
Because while you were thinking about learning,
you learned something.
And while you were learning something,
you thought about learning.
And while thinking about that,
you learned that you had been thinking about it.
Which is useful.
Probably.
But the book is still open

to the same page.